

Reviewer Pack — Operator Norm Separation for Read-Twice vs Read-Once BPs

Entry ce017a87b91d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 13:33:37 UTC

Operator Norm Separation for Read-Twice vs Read-Once BPs

Entry ID: ce017a87b91d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 13:33:37 UTC

1. Conjecture

Field A (mathematical branch): Operator Theory **Field B** (complexity object): Branching Programs

Statement:

For any read-once branching program P over n variables, the operator norm of its communication matrix M_P is $O(\log n)$. For any read-twice branching program Q computing the same function, $\|M_Q\|_{\text{op}} \geq \Omega(n)$.

Rationale (proposer's reasoning):

Operator norms capture structural differences in communication complexity, with read-once BPs constrained by logarithmic spectral properties while read-twice BPs exhibit linear growth due to repeated state access. This mirrors discrepancy gaps but through spectral analysis.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8cd0ed0da41fc864

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from typing import List, Dict

def gaussian_elimination(A: List[List[float]]) -> List[List[float]]:
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below the pivot
        factor = A[i][i]
        for j in range(i, n):
            A[i][j] /= factor
        for k in range(i+1, n):
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
    return A

def operator_norm(matrix: List[List[float]]) -> float:
    U, _, Vt = gaussian_elimination([row + [0] * (len(matrix) - len(row)) for row in matrix])
    singular_values = [math.sqrt(U[i][i]**2 + Vt[i][i]**2) for i in range(len(matrix))]
    return max(singular_values)

def generate_read_once_bp(n: int) -> List[List[int]]:
    bp = [[0] * (1 << n) for _ in range(1 << n)]
    for i in range(1 << n):
        for j in range(1 << n):
            if i & j == 0:
                bp[i][j] = random.randint(0, 1)
    return bp

def generate_read_twice_bp(n: int) -> List[List[int]]:
    bp = [[0] * (1 << n) for _ in range(1 << n)]
    for i in range(1 << n):
        for j in range(1 << n):
            if i & j == 0:
                bp[i][j] = random.randint(0, 1)
                bp[j][i] = bp[i][j]
    return bp
```

```

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        M_ro = generate_read_once_bp(n)
        M_twice = generate_read_twice_bp(n)

        norm_ro = operator_norm(M_ro)
        norm_twice = operator_norm(M_twice)

        results.append({
            "n": n,
            "norm_ro": norm_ro,
            "norm_twice": norm_twice
        })

    metric_value = sum(result["norm_ro"] for result in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(norm_ro <= math.log(n) and norm_twice >= n / 2 for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Operator Norm",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd64fle.py", line 98, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd64fle.py", line 70, in run_trial
    norm_ro = operator_norm(M_ro)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd64fle.py", line 39, in operator_norm
    U, _, Vt = gaussian_elimination([row + [0] * (len(matrix) - len(row)) for row in matrix])
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd64fle.py", line 31, in gaussian_elim
    A[i][j] /= factor
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with division by zero error, preventing result collection | next: Fix numerical stability in Gaussian elimination implementation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47239
2	propose	ollama_remote	qwen3:8b	0	0	58739
3	preregistration	ollama_remote	qwen3:8b	0	0	24195
4	novelty	ollama_remote	qwen3:8b	0	0	21027
5	novelty	ollama_remote	qwen3:8b	0	0	15238
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13592
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11567
8	judge	ollama_remote	qwen3:8b	0	0	13317

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 204914 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/ce017a87b91d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce017a87b91d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ce017a87b91d.tar.gz` (if generated)